PESSY - REPORTE DE ARQUITECTURA Y ESCALABILIDAD

Plan de Escalabilidad a 50,000 Usuarios en 1 Año

Proyecto: PESSY - Plataforma Digital de Identidad de Mascotas
GCP Project ID: gen-lang-client-0123805751 / polar-scene-488615-i0
Fecha: Marzo 26, 2026
Equipo: AI + Cloud Architecture
Región Hosting: US Central

---

1. ARQUITECTURA ACTUAL

1.1 Diagrama de Componentes

```
┌──────────────────────────────────────────────────────────────┐
│                      FRONTEND (Web + Mobile)                 │
│  ┌──────────────────────────────────────────────────────┐   │
│  │ React 18 + TypeScript + Tailwind CSS + Capacitor      │   │
│  │ - PWA con Service Worker                              │   │
│  │ - Offline support + App Shell Architecture            │   │
│  │ - Firebase Auth (Realtime Client SDK)                 │   │
│  │ - Vite build system (dist/ ~2.5 MB gzip)             │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                              ↓ HTTPS
┌──────────────────────────────────────────────────────────────┐
│                   FIREBASE HOSTING + CDN                     │
│  ┌──────────────────────────────────────────────────────┐   │
│  │ - 5 targets deployados (app, appqa, appit, etc)       │   │
│  │ - Assets cachea 1 año (immutable)                      │   │
│  │ - HTML/SW sin cache (no-cache headers)                 │   │
│  │ - Security headers (CSP, X-Frame-Options, HSTS, etc)   │   │
│  │ - Bandwidth incluído, CDN global                       │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                              ↓ REST API
┌──────────────────────────────────────────────────────────────┐
│              FIREBASE BACKEND (us-central1)                  │
│  ┌──────────────────────────────────────────────────────┐   │
│  │              CLOUD FUNCTIONS (Node.js 22)              │   │
│  │  ┌──────────────────────────────────────────────┐     │   │
│  │  │ FUNCTIONS EXPORTADAS (27 total):              │   │   │
│  │  │                                               │   │   │
│  │  │ EMAIL & ENGAGEMENT:                           │   │   │
│  │  │ - pessySendInvitationEmail                    │   │   │
│  │  │ - pessySendWelcomeEmail                       │   │   │
│  │  │ - pessySendCoTutorInvitation                  │   │   │
│  │  │ - onUserCreatedSendWelcome (auth trigger)     │   │   │
```

```
│  │  │ - sendGmailSyncConsentReminders (pub/sub daily) │  │  │
│  │  │                                                 │  │  │
│  │  │ NOTIFICATIONS (Push + Scheduling):              │  │  │
│  │  │ - sendScheduledNotifications (5min cron)        │  │  │
│  │  │ - sendDailyCareSummary (pub/sub daily)          │  │  │
│  │  │ - sendBroadcastPushCampaigns (pub/sub daily)    │  │  │
│  │  │ - cleanupOldNotifications (pub/sub daily)        │  │  │
│  │  │                                                 │  │  │
│  │  │ CLINICAL AI + DOCUMENT PROCESSING:              │  │  │
│  │  │ - analyzeDocument (HTTP callable)               │  │  │
│  │  │ - generateClinicalSummary (HTTP callable)        │  │  │
│  │  │ - resolveBrainPayload (HTTP callable)            │  │  │
│  │  │ - ingestHistory (exported, AI processing)        │  │  │
│  │  │                                                 │  │  │
│  │  │ GMAIL INGESTION (OAuth + Background):           │  │  │
│  │  │ - gmailIngestSession (HTTP callable)             │  │  │
│  │  │ - processGmailJob (pub/sub queue processor)      │  │  │
│  │  │ - clinicalAiAnalysis (background processing)     │  │  │
│  │  │                                                 │  │  │
│  │  │ COMPLIANCE + DATA:                              │  │  │
│  │  │ - submitDataDeletionRequest (HTTP)               │  │  │
│  │  │ - reconcileExistingTreatments (pub/sub)          │  │  │
│  │  │ - recomputeClinicalAlertsDaily (pub/sub)         │  │  │
│  │  │ - approveAccessRequest (HTTP)                    │  │  │
│  │  │                                                 │  │  │
│  │  │ MEDIA:                                          │  │  │
│  │  │ - uploadPetPhoto (HTTP callable, 60s timeout)    │  │  │
│  │  │ - sendCoTutorInvite (HTTP callable)              │  │  │
│  │  │                                                 │  │  │
│  │  └─────────────────────────────────────────────────┘  │  │
│  │                                                          │  │
│  │  RUNTIME SPECS:                                          │  │
│  │  - Memory: 512MB (uploadPetPhoto), default others     │  │
│  │  - Timeout: 60s (uploadPetPhoto), others variable     │  │
│  │  - Region: us-central1                                │  │
│  │  - Concurrency: 80 (default), varies by tier          │  │
│  │                                                          │  │
│  │  DEPENDENCIES:                                           │  │
│  │  - firebase-admin ^11.11.1                            │  │
│  │  - firebase-functions ^5.1.1                          │  │
│  │  - @google-cloud/vertexai ^1.10.0 (Gemini API)       │  │
│  │  - resend ^6.9.3 (email service)                      │  │
│  │  - mammoth ^1.11.0 (DOCX parsing)                     │  │
│  │  - Node.js 22 runtime                                 │  │
│  └──────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
        ↓                          ↓                        ↓
┌──────────────────┐  ┌──────────────────────┐  ┌──────────────────┐
│  FIRESTORE DB    │  │  CLOUD STORAGE       │  │  VERTEX AI       │
│                  │  │                      │  │  (Google Gemini)│
│ Collections:     │  │ Paths:               │  │                  │
│ - users          │  │ - users/{id}/**      │  │ Models:          │
```

```
| - pets            |  | - documents/{id}/**  |  | - gemini-2.0     |
| - medical_events  |  |                      |  | - textembedding  |
| - clinical_*      |  | Max file: 30MB       |  | - speech-to-     |
| - medications     |  | (per storage.rules)  |  |   text           |
| - treatments      |  |                      |  |                  |
| - appointments    |  | Auth: User-scoped    |  | Datastore:       |
| - clinical_*      |  |                      |  | RAG (optional)   |
| - reminders       |  |                      |  |                  |
| - dose_events     |  |                      |  | Rate limits:     |
| - verified_*      |  |                      |  | Check Quotas     |
| - invitations     |  |                      |  |                  |
| - access_*        |  |                      |  | Tokens:          |
| - gmail_*_tmp     |  |                      |  | ~2K per request  |
| - scheduled_*     |  |                      |  | (clinical docs)  |
|                   |  |                      |  |                  |
| Indexes: 9        |  |                      |  | Cost:            |
| (see 1.3)         |  |                      |  | Pay-per-call     |
└──────────────────┘  └──────────────────────┘  └──────────────────┘
```

1.2 Servicios Firebase en Uso vs Configurados

Servicio	Usado	Config	Notas
Firebase Auth	✅ Sí	✅ Sí	Email + Custom providers, ID tokens
Firestore DB	✅ Sí	✅ Sí	~20 colecciones, security rules complejas
Cloud Storage	✅ Sí	✅ Sí	users/{id} + documents/{id} paths
Cloud Functions	✅ Sí	✅ Sí	27 functions, Node.js 22, ~20K LOC
Hosting	✅ Sí	✅ Sí	5 targets (prod, QA, IT, etc)
FCM (Push)	✅ Sí	✅ Sí	Browser + Mobile (Capacitor)
Vertex AI API	✅ Sí	✅ Sí	Gemini 2.0 para análisis clínico
Pub/Sub	✅ Sí	✅ Sí	Scheduled tasks (Cloud Scheduler trigger)
Cloud Scheduler	✅ Sí	✅ Sí	Daily + 5min cron jobs
Realtime DB	❌ No	❌ No	Usar Firestore en su lugar
Cloud Tasks	❌ No	❌ No	Podría optimizar queue de emails
Datastore	⚠️ Optional	❌ No	RAG vector store para Gemini (experimental)
Cloud SQL	❌ No	❌ No	No necesario, Firestore es suficiente
Bigtable	❌ No	❌ No	No necesario en esta escala
BigQuery	❌ No	❌ No	Logs en Cloud Logging en su lugar

1.3 Cloud Functions - Inventario Detallado

Total LOC: ~19,951 líneas TypeScript **Archivos:** 18 .ts files **Memoria:** 512MB (uploadPetPhoto), default para otros **Timeout:** 60s (uploadPetPhoto), variable para otros **Concurrency:** 80 (default limit)

Función	Tipo	Memory	Timeout	Trigger	
uploadPetPhoto	HTTP	512MB	60s	https.onCall	S r S
pessySendInvitationEmail	HTTP	256MB	30s	https.onCall	E c (
pessySendWelcomeEmail	HTTP	256MB	30s	https.onCall	E b u
pessySendCoTutorInvitation	HTTP	256MB	30s	https.onCall	C i
onUserCreatedSendWelcome	Auth	256MB	30s	auth.user().onCreate	V e r
gmailIngestSession	HTTP	1GB	540s	https.onCall	I C
processGmailJob	Pub/Sub	1GB	540s	pubsub topic	P b e
sendScheduledNotifications	Pub/Sub	512MB	120s	Cloud Scheduler (5m)	N p
sendDailyCareSummary	Pub/Sub	512MB	120s	Cloud Scheduler (daily)	E r c
sendBroadcastPushCampaigns	Pub/Sub	512MB	120s	Cloud Scheduler (daily)	P c u
sendGmailSyncConsentReminders	Pub/Sub	256MB	60s	Cloud Scheduler (daily)	P a C
reconcileExistingTreatments	Pub/Sub	512MB	300s	Cloud Scheduler (daily)	S t
recomputeClinicalAlertsDaily	Pub/Sub	512MB	300s	Cloud Scheduler (daily)	P a c
cleanupOldNotifications	Pub/Sub	256MB	60s	Cloud Scheduler (daily)	L r

analyzeDocument	HTTP	512MB	120s	https.onCall	
generateClinicalSummary	HTTP	512MB	120s	https.onCall	
resolveBrainPayload	HTTP	256MB	30s	https.onCall	
ingestHistory	Exported	512MB	120s	Función lib	
submitDataDeletionRequest	HTTP	256MB	30s	https.onCall	
approveAccessRequest	HTTP	256MB	30s	https.onCall	
sendCoTutorInvite	HTTP	256MB	30s	https.onCall	

Estadísticas:

- Funciones HTTP Callable: 12
- Funciones Pub/Sub (scheduled): 8
- Funciones Auth trigger: 1
- Funciones exported (library): 1

1.4 Flujo de Datos - Documento Escaneado

```
1. USUARIO ESCANEA DOCUMENTO (Frontend)
   ├─ Abre camera/file picker
   ├─ Captura imagen (JPEG/PNG, ~2-5 MB)
   └─ Selecciona tipo (Vacuna/Análisis/Receta/General)

2. FRONTEND → CLOUD FUNCTIONS
   └─ Llamada: analyzeDocument(imageBase64, type, petId)

3. CLOUD FUNCTION (analyzeDocument)
   ├─ Valida auth (request.auth.uid)
   ├─ Sube imagen a Storage: documents/{userId}/tmp/{uuid}
   ├─ Llama Vertex AI / Gemini API
   │  └─ Prompt: "Extrae info médica del documento"
   │  └─ Response: { campos extraídos + confianza }
   ├─ Almacena resultado en Firestore:
   │  └─ /clinical_events/{eventId}
```

```
   │  └─ /clinical_review_drafts/{draftId}
   └─ Retorna: { extractedData, confidenceScores }

4. FRONTEND MUESTRA RESULTADOS
   ├─ Preview de campos extraídos
   ├─ Editor inline para corregir
   └─ Botón "Guardar" → actualiza Firestore

5. FIRESTORE ACTUALIZADO
   └─ Dispara Cloud Function (trigger): recomputeClinicalAlertsDaily
      └─ Re-calcula alertas clínicas por mascota

ALMACENAMIENTO TOTAL:
├─ Imagen original: ~3 MB/doc
├─ Metadata en Firestore: ~1 KB
└─ Embeddings (si usa RAG): ~2 KB
```

1.5 Flujo de IA - Integración Gemini

```
┌─ FRONTEND (React)
│  └─ analyzeDocument(base64Image, type, petId)
│
└─> CLOUD FUNCTION (Node.js 22)
   ├─ Init Vertex AI client:
   │  └─ new VertexAI({
   │       project: "gen-lang-client-0123805751",
   │       location: "us-central1"
   │     })
   │
   ├─ Prepare request:
   │  ├─ Model: "gemini-2.0-flash" (fast) o "gemini-pro" (quality)
   │  ├─ Tokens: ~2000 por request
   │  └─ Temperatura: 0.1 (determinístico)
   │
   ├─ System Prompt (tailored por tipo):
   │  └─ "Eres experto en análisis de documentos clínicos veterinarios..."
   │  └─ "Extrae: nombre, fechas, dosis, veterinario, observaciones"
   │  └─ "Retorna JSON estructurado con confianza por campo"
   │
   ├─ Submit to Gemini API:
   │  └─ generateContent({
   │       contents: [{ role: "user", parts: [imageData, prompt] }]
   │     })
   │
   ├─ Parse response:
   │  └─ JSON: { vacuneName, date, veterinarian, confidence: 0.95 }
   │
   └─> STORE in Firestore
      └─ /clinical_events/{petId}/clinical_events/{docId}
      └─ Fields: { extractedData, aiModel, timestamp, confidence }
```

Endpoints Gemini Usados:

- `vertexai.getGenerativeModel("gemini-2.0-flash")`
- `model.generateContent()` — Análisis de imágenes/texto
- Opcional: `model.createEmbedding()` — Para RAG/búsqueda vectorial

Estimación de Tokens:

- Image + prompt: ~1,000-2,000 tokens
- Response: ~500-1,000 tokens
- **Total: ~2,500 tokens/request máximo**

---

2. EVALUACIÓN DE ESCALABILIDAD A 50K USUARIOS

2.1 Estructura de Datos Firestore

Proyección para 50K usuarios:

```
users/
├─ {userId}/
│  ├─ user metadata (name, email, phone) — ~1 KB
│  ├─ pets/{petId} — reference (1 per pet)
│  ├─ fcm_tokens/{tokenId} — ~200 B cada (3-5 por usuario)
│  └─ Tamaño por usuario: ~3 KB

pets/
├─ {petId}/
│  ├─ basic info (name, breed, DOB, weight, microchip) — ~500 B
│  ├─ medical data (allergies, conditions) — ~1 KB
│  ├─ ownerId, coTutorUids — ~100 B
│  └─ Tamaño por mascota: ~2 KB
├─ Relación: ~1.5 mascotas/usuario
├─ Total mascotas: 75,000

medical_events/ (vaccine, procedure, etc)
├─ {eventId}/
│  ├─ petId, type, date, notes, veterinarian — ~800 B
│  └─ Tamaño por evento: ~800 B
├─ Estimado: ~5 eventos/mascota
├─ Total: 375,000 eventos

clinical_events/ (diagnoses, analysis)
├─ Tamaño similar: ~1 KB
├─ Estimado: ~3/mascota
├─ Total: 225,000 eventos

medications/ (ongoing meds)
├─ Tamaño: ~600 B
├─ Estimado: ~2/mascota
├─ Total: 150,000

treatments/ (therapy records)
```

```
├─ Tamaño: ~800 B
├─ Estimado: ~4/mascota
├─ Total: 300,000

appointments/ (future/past vet visits)
├─ Tamaño: ~700 B
├─ Estimado: ~10/mascota
├─ Total: 750,000

scheduled_notifications/
├─ Tamaño: ~600 B
├─ Estimado: ~5 activas/usuario
├─ Total: 250,000

reminders/ + dose_events/ + scheduled_reminders/
├─ Tamaño combinado: ~500 B
├─ Estimado: ~8/usuario
├─ Total: 400,000

verified_reports/
├─ Tamaño: ~1.5 KB
├─ Estimado: ~0.5/usuario
├─ Total: 25,000

invitations/ (co-tutor links)
├─ Tamaño: ~300 B
├─ Estimado: ~0.1/usuario (inactive)
├─ Total: 5,000

Otros (clinical_conditions, diagnoses, treatments, clinical_alerts, etc)
├─ Total estimado: ~500,000 documentos
```

TOTAL DOCUMENTOS FIRESTORE: ~3.5 millones TAMAÑO PROMEDIO: ~700 B TAMAÑO TOTAL ESTIMADO: ~2.4 TB

2.2 Firestore - Capacidad de Escala

Métrica	Límite Google	Proyección 50K	Status
Documentos	5M+	3.5M	✅ OK
Tamaño DB	10TB	2.4 TB	✅ OK
Escrituras/seg	100K	~100-200/s	✅ OK
Lecturas/seg	500K+	~500-1000/s	✅ OK
Tamaño doc	1 MB max	~1 KB avg	✅ OK
Índices	200+	9 current	✅ OK
Sharding	Automático	Recomendado	⚠️ Revisar

Evaluación: ✅ Firestore ESCALA bien a 50K usuarios

Índices Necesarios (ya configurados):

```
{
  "scheduled_notifications": [active, sent, scheduledFor],
  "broadcast_push_campaigns": [active, sent, sendAt],
  "gmail_ingestion_sessions": [status, updated_at],
  "scheduled_reminders": [status, scheduledFor],
  "dose_events": [status, dueAt],
  "treatments": [subtype, generatedFromEventId, petId],
  "access_requests": [status, createdAt DESC]
}
```

Sharding Recomendado para:

- `scheduled_notifications` (250K docs, alta concurrencia)
- `medical_events` (375K docs, escritura frecuente)
- `treatments` (300K docs)

2.3 Cloud Storage - Proyección

Tipos de archivos:

1. **Fotos de mascotas** (~1 MB cada, comprimidas)
   - ~1 foto/mascota
   - 75K mascotas = 75 GB
2. **Documentos médicos escaneados** (~3 MB original, ~500KB comprimido)
   - ~5 docs/mascota
   - 375K docs = 187.5 GB
3. **PDFs generados** (~200 KB cada)
   - ~0.5 reportes/usuario (cacheados)
   - 25K reportes = 5 GB

TOTAL STORAGE: ~267.5 GB (sin redundancia)

Métrica	Límite	Proyección	Status
Almacenamiento	Ilimitado	267 GB	✅ OK
Tamaño archivo	5 TB	3 MB avg	✅ OK
Ancho banda descarga	Ilimitado (pago)	~50-100 GB/día	⚠️ Costos
Ancho banda subida	Ilimitado (pago)	~20-30 GB/día	⚠️ Costos
Operaciones/día	Millones	200K+	✅ OK

Storage Lifecycle (recomendado):

- Archivos tmp: eliminar después 7 días
- Documentos old (>2 años): mover a archivos

- PDFs cached: eliminar después 30 días

2.4 Firebase Auth - Límites

Métrica	Límite	Proyección 50K	Status
Usuarios registrados	Ilimitado	50,000	✅ OK
Sesiones simultáneas	5/segundo	~1-5/s	✅ OK
ID tokens/seg	10K	~10-20/s	✅ OK
OAuth providers	5+	Email + 0 other	✅ OK
Custom claims	1 KB	Not used	✅ OK
Refresh token rotate	Daily	Recomendado	✅ OK

Recomendación:

- Implementar **email verification** obligatorio
- Usar **password reset flows** estándares
- Considerar **multi-factor auth** (2FA) para admins

2.5 FCM (Firebase Cloud Messaging) - Proyección

Parámetros:

- Usuarios: 50,000
- % Con notificaciones activas: 70% = 35,000
- Notificaciones por usuario/día: 2-3
- **Total mensajes/día: ~100,000 FCM messages**

Métrica	Límite	Proyección	Status
Mensajes/segundo	10M+	~1,200/s	✅ OK
Mensajes/día	Ilimitado	100K	✅ OK
Topicos	Ilimitado	~100	✅ OK
Subscribers/topico	Ilimitado	100-10K	✅ OK
Tamaño mensaje	4 KB	1-2 KB	✅ OK
Tasa entrega	99.9%	Recomendado 95-98%	✅ OK

Tipos de notificaciones esperadas:

- Recordatorio vacuna próxima: 500-1K/día
- Recordatorio medicamento: 1K-2K/día
- Recordatorio cita veterinaria: 500-1K/día
- Broadcast campaigns: 2-5K/día
- Transaccionales: 1K-5K/día

2.6 Cloud Functions - Evaluación de Escala

Proyección de invocaciones/mes para 50K usuarios:

Función	Invocaciones/mes	Peak/segundo	Timeout	Cos Es
analyzeDocument	375K (5/user/mes)	5	120s	$18,7
uploadPetPhoto	75K (1/user/año)	1	60s	$4,50
sendScheduledNotifications	3M (5min cron)	50	120s	$15,0
sendDailyCareSummary	50K (daily)	1	120s	$2,50
sendBroadcastPushCampaigns	50K (daily)	1	120s	$2,50
sendGmailSyncConsentReminders	50K (daily)	1	60s	$2,00
gmailIngestSession	5K (0.1/user)	0.5	540s	$5,00
processGmailJob	50K-100K (async)	10	540s	$20,0
recomputeClinicalAlertsDaily	50K (daily)	1	300s	$2,50
reconcileExistingTreatments	50K (daily)	1	300s	$2,50
Otros (HTTP + triggers)	100K	5	30-60s	$5,00
			TOTAL/mes:	**~$80**

Cold Start Impact:

- Estimated cold starts: ~10-20% of invocations
- Cold start latency: +1-3 segundos
- **Impacto en UX:** Aceptable (OCR ya toma 5-10s)

Concurrency Limits (por región):

- Default: 80 concurrent executions
- Peak projected: 50 concurrent (analyzeDocument + email batch)
- **Action:** Request limit increase to 200 if needed

2.7 Hosting - Bandwidth y Performance

CDN Global de Firebase:

- Locations: 140+ POPs (worldwide)
- Cache: assets (1 year), HTML (no-cache)

Proyección de Bandwidth:

- Daily Active Users (DAU): 50K * 40% = 20K
- Sessions/user/day: 2
- Requests/session: ~15
- **Total requests/day: 600K**

Métrica	Proyección	Costo Firebase
Bundle JS/CSS	2.5 MB * 600K = 1.5 TB	$0.12/GB = $180
Assets (cached 1yr)	500 MB * 100K = 50 GB	$6
Imágenes dinámicas	1 MB * 300K = 300 GB	$36
HTML/SW (no-cache)	100 KB * 600K = 60 GB	$7
Total bandwidth/mes	~1.9 TB	~$229

---

3. PROYECCIÓN DE COSTOS PARA 50K USUARIOS

3.1 Desglose de Costos Mensuales

```
FIRESTORE DATABASE
├─ Storage: 2.4 TB * $0.18/GB = $432
├─ Reads: 50M/mes * $0.06/100K = $300
├─ Writes: 15M/mes * $0.18/100K = $27
├─ Deletes: 5M/mes * $0.02/100K = $1
└─ Subtotal Firestore: $760

CLOUD STORAGE
├─ Storage: 267 GB * $0.020/GB = $5.34
├─ Downloads: 50 GB/día * 30 * $0.12/GB = $180
├─ Uploads: 20 GB/día * 30 * $0.12/GB = $72
└─ Subtotal Storage: $257

CLOUD FUNCTIONS
├─ Invocations: 5.5M/mes * $0.40/1M = $2.20
├─ Compute time: 1M vCPU-sec * $0.000025/vCPU-sec = $25
├─ GB-seconds: 100K GB-sec * $0.0000041 = $0.41
├─ Outbound data: 50 GB * $0.12 = $6
└─ Subtotal Functions: $34 (muy barato!)

FIREBASE HOSTING
├─ Storage: 5 GB * $0.018/GB = $0.09
├─ Bandwidth: 1.9 TB * $0.12/GB = $228
└─ Subtotal Hosting: $228

FIREBASE AUTH
├─ Users: 50K (first 50K FREE)
└─ Subtotal Auth: $0

FCM (PUSH NOTIFICATIONS)
├─ Messages: 100K/day * 30 = 3M/mes
└─ Subtotal FCM: $0 (INCLUIDO en Firebase)

VERTEX AI / GEMINI API
├─ Input tokens: 375K docs * 2K tokens = 750M tokens
```

```
│   └─ $0.075 / 1M input tokens = $56.25
├─ Output tokens: 375K docs * 1K tokens = 375M tokens
│   └─ $0.30 / 1M output tokens = $112.50
└─ Subtotal Vertex AI: $169

CLOUD LOGGING (logs, monitoring)
├─ Log ingestion: ~10 GB/mes * $0.50/GB = $5
├─ Log storage: free (90 días default)
└─ Subtotal Logging: $5

CLOUD TASKS (optional, si se implementa)
├─ Task invocations: 1M/mes * $0.40/1M = $0.40
└─ Subtotal Tasks: $0.40

CLOUD SCHEDULER (cron jobs)
├─ Jobs: 10 jobs * $0.10 = $1
└─ Subtotal Scheduler: $1

────────────────────────────────────
TOTAL MENSUAL ESTIMADO: $1,455
────────────────────────────────────

PER USER METRICS (50K users):
├─ Costo/usuario/mes: $1,455 / 50K = $0.029
├─ Costo/usuario/año: $0.35
└─ A escala: RENTABLE si precio ≥ $0.99/mes
```

3.2 Comparación con Free Tiers

Firebase Free Tier (Spark):

```
✅ INCLUIDO:
├─ Firestore: 1 GB storage + 50K reads/día + 20K writes/día
├─ Storage: 5 GB
├─ Functions: 125K invocations/mes, 40K GB-sec/mes
├─ Hosting: 10 GB storage + 360 MB bandwidth/día
├─ Auth: 50K users (FREE, sin limites)
└─ FCM: GRATIS (unlimited)

❌ NO INCLUIDO:
├─ Vertex AI / Gemini API
├─ Cloud Logging (limited)
├─ Cloud Scheduler (limited)
└─ Cloud Tasks
```

Con 50K usuarios, SUPERA free tier en:

- ✅ Firestore storage (2.4 TB vs 1 GB)
- ✅ Firestore read/write volumen (50M reads vs 50K/día)
- ✅ Storage bandwidth
- ✅ Cloud Functions
- ✅ Hosting bandwidth

RECOMENDACIÓN: Cambiar a Blaze Plan (pago) desde ~1K usuarios

---

3.3 Optimizaciones para Reducir Costos

Optimización	Ahorro Mensual	Esfuerzo	Impacto
CDN con Redis (caching cliente)	-$50	Alto	Reducir reads
Batch document analysis	-$30	Medio	Reducir Vertex calls
Sharding usuarios activos	-$20	Medio	Reducir reads
Image compression	-$40	Bajo	Reducir storage
Scheduled cleanup	-$15	Bajo	Eliminar datos viejos
Reduce log verbosity	-$5	Bajo	Menos logs
Implement caching layer	-$100	Alto	Reducir reads + writes

POTENCIAL AHORRO: $260/mes (-18%)

---

4. PROGRAMAS DE CRÉDITOS DE GOOGLE

4.1 Google for Startups - Cloud Credits

Programa: Google Cloud for Startups
Créditos: $3,000-$10,000 USD
Duración: 12 meses
Requisitos:

- Ser startups (< 5 años, < $5M funding)
- Aplicar a través de partner (Y Combinator, Techstars, etc)
- O aplicar directamente en google.com/startups

Cómo solicitar:

1. Ir a: https://cloud.google.com/startup/docs/credits
2. Seleccionar programa (Google for Startups, Firebase Spark to Blaze, etc)
3. Llenar aplicación con:
    - Business plan
    - Team info
    - Product description
    - Why Google Cloud

Duración: 1-2 semanas para aprobación

4.2 Firebase Spark → Blaze Transition Credits

Programa: Firebase Blaze Onboarding
Créditos: $12 crédito inicial en Blaze
Duración: Aplicable de inmediato
Requisitos:

- Convertir de Spark a Blaze plan
- Agregar método de pago
- Link de referencia (opcional)

Cómo acceder:

1. Firebase Console → Project settings
2. Billing → Upgrade to Blaze
3. $12 crédito aplicado automático

4.3 GCP Free Trial

Programa: Google Cloud Free Trial
Créditos: $300 USD
Duración: 90 días
Requisitos:

- Tarjeta de crédito válida
- Cuenta Google nueva (no hay free trial previo)
- Billing account nuevo

Cómo solicitar:

1. console.cloud.google.com
2. Click "Try for free"
3. Seguir flujo de registro

Límites:

- Algunos servicios tienen cuotas adicionales (e.g., Vertex AI)
- Después de 90 días, requiere conversión a plan pago

4.4 Estrategia de Créditos Recomendada para PESSY

```
FASE 1 (Startup Launch - 0-3 meses):
├─ GCP Free Trial: $300 crédito (90 días)
├─ Firebase Spark plan: $0 (dentro de free tier)
└─ Costo real: $0

FASE 2 (Growth - 3-6 meses):
├─ Aplicar a Google for Startups
├─ Solicitar $5,000 crédito (12 meses)
├─ Firebase Blaze: +$12 crédito
├─ Proyección costo: $1,455/mes * 3 = $4,365
└─ Crédito cubre 100% durante 3 meses

FASE 3 (Scale - 6-12 meses):
├─ Usar créditos restantes: $1,635
├─ Costo adicional: $1,455/mes * 6 meses = $8,730
├─ Pago directo: $8,730 - $1,635 = $7,095
├─ Promedio/mes: $1,455 (PAGO)
└─ Suscribir a SaaS partners para descuentos

FASE 4 (Post-Launch - 12+ meses):
```

```
├─ Créditos agotados
├─ Costo mensual: ~$1,455
├─ Buscar sponsorship o VC funding
└─ Negociar descuentos volumen con Google
```

RESUMEN:

- **Primeros 12 meses:** ~$5,700 en créditos (95% coverage)
- **Costo real aprobado:** ~$345
- **Rentabilidad:** Viabilidad a $0.99/mes/usuario

---

5. RECOMENDACIONES DE ARQUITECTURA

5.1 Cambios INMEDIATOS (Antes de Scale)

1. Automatizar Deployment (CI/CD)

Problema: Manual deployments → riesgo de stale builds **Solución:**

```yaml
# .github/workflows/deploy.yml
name: Deploy to Firebase

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Build Frontend
        run: npm run build

      - name: Deploy Functions
        run: npm --prefix functions run deploy

      - name: Deploy Hosting + Firestore
        uses: FirebaseExtended/action-hosting-deploy@v0
        with:
          repoToken: ${{ secrets.GITHUB_TOKEN }}
          firebaseServiceAccount: ${{ secrets.FIREBASE_KEY }}
          projectId: polar-scene-488615-i0
```

Beneficio: 0 downtime, rollback automático, audit trail

---

2. Implementar Caching Strategy

Problema: Reads de Firestore = costos altos **Solución (3 niveles):**

```
// Level 1: Browser localStorage (offline, <5MB)
localStorage.setItem(`pet_${petId}`, JSON.stringify(petData));

// Level 2: Service Worker cache (long-lived, 50MB)
caches.open('pessy-v1').then(cache => {
  cache.put(`/api/pet/${petId}`, response);
});

// Level 3: Server-side Redis/Memcache (distributed)
// Usar Cloud Memorystore for Redis (Firebase doesn't have)
// OR: Implement cache in Cloud Functions:
const cache = new Map(); // Simple in-memory
```

Estimado Ahorro: 30-40% en Firestore reads

---

3. Imagen Compression Pipeline

Problema: Documentos escaneados = 3-5 MB cada **Solución:**

```
// En Cloud Function: analyzeDocument
import sharp from 'sharp';

const compressed = await sharp(imageBuffer)
  .resize(1280, 960, { fit: 'inside' })
  .jpeg({ quality: 75, progressive: true })
  .toBuffer();
// Result: 3 MB → 200-400 KB
```

Beneficio: Reducir storage + bandwidth ~80%

---

4. Implementar Rate Limiting

Problema: Abuse prevention, cost control **Solución:**

```
// Firestore security rules
match /clinical_events/{eventId} {
  allow create: if canAccessPetById(request.resource.data.petId)
    && rateLimitCheck(request.auth.uid, "create_clinical_event", 10, 3600);
    // Max 10 creations per hour per user
}

// Cloud Function helper:
function rateLimitCheck(userId, action, limit, windowSeconds) {
  const key = `${userId}:${action}`;
  const count = getFromCache(key) || 0;
  if (count >= limit) return false;
  setInCache(key, count + 1, windowSeconds);
```

```
  return true;
}
```

---

5. Security Hardening

Implementar AHORA:

```
// firestore.rules — Actualizar:

// 1. Validate request size
allow create, update: if request.resource.size < 100000; // 100KB max

// 2. Rate limit per user per day
allow create: if getMonthlyCallCount() < 1000;

// 3. Timestamp validation
allow create: if request.resource.data.createdAt == request.time;

// 4. Email verification required for sensitive ops
allow update: if request.auth.token.email_verified == true;

// 5. Implement admin-only collections
match /audit_logs/{logId} {
  allow read, create: if request.auth.token.email == 'admin@pessy.app';
}
```

5.2 Optimizaciones Arquitectónicas

1. Implementar Queue System para Heavy Workloads

Problema: Gmail ingestion (processGmailJob) puede timeout a 540s

Solución: Cloud Tasks + Pub/Sub

```
// En Cloud Function: gmailIngestSession
import { CloudTasksClient } from '@google-cloud/tasks';

async function createTaskForGmailProcessing(userId) {
  const tasksClient = new CloudTasksClient();
  const parent = tasksClient.queuePath(PROJECT, LOCATION, 'gmail-queue');

  const task = {
    httpRequest: {
      httpMethod: 'POST',
      url: `https://us-central1-${PROJECT}.cloudfunctions.net/processGmailJob`,
      headers: { 'Content-Type': 'application/json' },
      body: Buffer.from(JSON.stringify({ userId })).toString('base64'),
    },
    scheduleTime: { seconds: Math.floor(Date.now() / 1000) },
  };
```

```
  await tasksClient.createTask({ parent, task });
}
```

Beneficio: Parallelizar 1000s de jobs, no saturar concurrency

---

2. Implementar Pub/Sub Pattern para Notifications

Actual: sendScheduledNotifications corre cada 5 min **Problema:** Spike de lecturas cada 5 min

Solución:

```
// Usar Firestore triggers + Pub/Sub directamente
export const onScheduledNotificationCreated = functions.firestore
  .document('scheduled_notifications/{notifId}')
  .onCreate(async (snap) => {
    const notif = snap.data();
    if (notif.active && notif.scheduledFor <= admin.firestore.Timestamp.now()) {
      // Publish directamente a FCM topic
      await publishToFCM(notif);
    }
  });

// Batching en Cloud Pub/Sub:
const publisher = new PubSub().topic('send-notifications');
const messageId = await publisher.publish(
  Buffer.from(JSON.stringify(notificationBatch))
);
```

Beneficio: Real-time delivery, mejor resource utilization

---

3. Implementar Vector Search para RAG (Gemini)

Problema: Búsqueda de documentos clínicos = fuzzy/lenta

Solución: Vertex AI Search

```
// En analyzeDocument: agregar embeddings
import { TextEmbeddingModel } from '@google-cloud/vertexai';

const embeddingModel = vertexai.getGenerativeModel({
  model: "text-embedding-004",
});

const embeddings = await embeddingModel.embedContent({
  content: {
    parts: [{ text: extractedMedicalData }],
  },
});

// Guardar embeddings en Firestore
await db.collection('clinical_events').doc(docId).set({
  text: extractedMedicalData,
```

```
  embedding: embeddings.embedding.values, // Vector [768]
  metadata: { petId, type, timestamp },
}, { merge: true });

// Luego: query vectorial
const queryEmbedding = await embeddingModel.embedContent({
  content: { parts: [{ text: "vaccine history for allergies" }] },
});

// Use Vertex Vector Search o: manual cosine similarity en Cloud Functions
```

Beneficio: Búsqueda semántica de documentos médicos, mejorar UX

---

4. Implement Progressive Web App (PWA) Caching

Actual: Service Worker simple **Mejorado:**

```
// public/sw.js — Workbox config
importScripts('https://cdn.jsdelivr.net/npm/workbox-cdn@7/workbox/workbox-sw.js');

workbox.routing.registerRoute(
  ({ request }) => request.destination === 'image',
  new workbox.strategies.CacheFirst({
    cacheName: 'pessy-images',
    plugins: [
      new workbox.expiration.ExpirationPlugin({
        maxEntries: 1000,
        maxAgeSeconds: 7 * 24 * 60 * 60, // 7 days
      }),
    ],
  })
);

workbox.routing.registerRoute(
  ({ request }) => request.destination === 'document',
  new workbox.strategies.NetworkFirst({
    cacheName: 'pessy-docs',
    networkTimeoutSeconds: 3,
  })
);

// Periodic sync para background uploads
self.addEventListener('periodicsync', (event) => {
  if (event.tag === 'sync-documents') {
    event.waitUntil(syncPendingDocuments());
  }
});
```

Beneficio: Funcionar offline 100%, mejor rendimiento

5.3 Monitoring y Alertas

Implementar Cloud Monitoring

```
// En Cloud Functions:
import { Monitoring } from '@google-cloud/monitoring';

const monitoring = new Monitoring.MetricServiceClient();

async function recordCustomMetric(value, label) {
  const request = {
    name: monitoring.projectPath(PROJECT),
    timeSeries: [{
      metric: {
        type: 'custom.googleapis.com/pessy/document_analyzed',
        labels: { 'document_type': label },
      },
      points: [{
        interval: { endTime: { seconds: Math.floor(Date.now() / 1000) } },
        value: { doubleValue: value },
      }],
    }],
  };
  await monitoring.createTimeSeries(request);
}
```

Alertas Recomendadas:

- Error rate > 1% en Cloud Functions
- Latencia p95 > 5 segundos
- Firestore reads > 100K/sec
- Storage bandwidth > 100 GB/día
- Vertex AI errors > 0.1%

---

6. BANDERAS ROJAS Y RIESGOS

6.1 Bottlenecks que Explotan a Escala

Bottleneck	Actual	50K Usuarios	Solución
Firestore cold reads	1,000/s	50,000/s	Sharding + caching
Cloud Functions concurrency	80	100+ peak	Aumentar límite + queue system
Gmail ingestion latency	540s	Timeout	Cloud Tasks + background processing
Image processing time	30s average	Cumulative delay	Compress + async queue

FCM delivery time	1-2s	Network congestion	Batch + prioritize
Vertex AI token usage	Low	750M/mes	Optimize prompts, cache results
Storage bandwidth	50 GB/día	Surge during peak	CDN caching layer

---

6.2 Costos Ocultos

Costo Oculto	Estimación	Impacto
Egress bandwidth (Europe users)	$0.12/GB	+$20-50/mes
Vertex AI overuse	Extra 1M tokens	+$75/mes
Pub/Sub message volume	500K msgs/día	+$200/mes if not optimized
Cloud Logging retention	90 días default	+$10/mes
Unused indexes	9 indexes * cost	-$5/mes (not critical)
Database backup	Automated snapshots	+$5-10/mes (hidden)
Domain registration	pessy.app	$12/año
SSL certificates	Firebase managed	$0 (included)
Email delivery (Resend API)	$0.50 per 1K emails	+$25/mes (1.5M/year)

Total Hidden Costs: ~$60-130/mes

6.3 Single Points of Failure

SPOF	Impacto	Mitigation
Firestore region (us-central1)	Outage = 100% downtime	Implementar Firestore replicas (multi-region)
Cloud Functions region	Función = downtime	Deploy a múltiples regiones, load balancer
Storage bucket	No uploads/downloads	Replicación automática incluída
Vertex AI quota	No analysis = feature broken	Implementar fallback a prompt local o cache
Firebase Auth	No login	Raro, pero sin workaround
Resend email service	No emails = user confusion	Implementar fallback (SendGrid, AWS SES)
Gmail OAuth token	No ingestion	Refresh token flow, error handling

Recomendación: Multi-region Cloud Functions, Firestore replicas

---

6.4 Vulnerabilidades de Seguridad

CRÍTICAS (implementar ya):

1. **Inyección SQL-like en Gemini prompts**

```
// ❌ MALO:
const prompt = `Extract data from ${userProvidedText}`;

// ✅ BUENO:
const prompt = `Extract data from:
[DOCUMENT]\n${sanitize(userProvidedText)}\n[/DOCUMENT]`;
```

2. **Token leakage en logs**

```
// ❌ MALO:
console.log(`Gmail token: ${gmailToken}`);

// ✅ BUENO:
console.log(`Gmail token: ${gmailToken.substring(0, 5)}***`);
```

3. **CORS misconfiguration**

```
// ❌ MALO:
headers: { 'Access-Control-Allow-Origin': '*' }

// ✅ BUENO:
headers: { 'Access-Control-Allow-Origin': 'https://pessy.app' }
```

4. **Rate limiting ausente**

```
// Implementar siempre en Cloud Functions:
if (invocationCount > RATE_LIMIT) {
  throw new functions.https.HttpsError('resource-exhausted', 'Rate limit exceeded');
}
```

5. **Data exposure en URLs**

```
// ❌ MALO:
window.location = `/share?data=${btoa(petData)}`;

// ✅ BUENO:
// Usar secure links con expiración:
const shareLink = await createSecureShareLink(petId, expiresIn);
```

6.5 Compliance - GDPR + Argentina (Ley 25.326)

Requerimientos legales:

```
GDPR (EU users):
├─ Right to deletion ✅ (data-deletion Cloud Function)
├─ Right to portability ❌ (implementar export JSON)
├─ Consent management ❌ (cookie banner)
├─ Data Processing Agreement ❌ (con Google Cloud)
└─ Data residency ❌ (data en US, EU may require EU)

Ley 25.326 (Argentina):
├─ Registro de BD ante APDP ❌ (tramitar)
├─ Consentimiento explícito ✅ (on signup)
├─ Derecho de acceso ❌ (implementar /api/export)
├─ Derecho a rectificación ✅ (edit profile)
├─ Derecho a supresión ✅ (data-deletion)
└─ Datos sensibles (salud) ⚠️ (clasificar, encriptar)
```

Acciones inmediatas:

1. Agregar "Data Processing Agreement" a Google Cloud
2. Implementar `/api/user/export` endpoint (GDPR right to data portability)
3. Clasificar pets como "sensitive personal data"
4. Encriptar datos de salud en tránsito y en reposo

---

7. PLAN DE ACCIÓN PRIORIZADO

Fase 0 (Ahora - 2 semanas)

- [ ] Implementar CI/CD (GitHub Actions)
- [ ] Agregar Cloud Monitoring + alertas
- [ ] Implementar image compression
- [ ] Rate limiting en Firestore rules
- [ ] Security headers review
- [ ] Solicitar Google for Startups crédito

Fase 1 (2-4 semanas)

- [ ] Implementar caching (localStorage + Service Worker)
- [ ] Vector embeddings para RAG search
- [ ] Cloud Tasks queue system
- [ ] GDPR data export endpoint
- [ ] Firestore sharding para hot collections

Fase 2 (4-8 semanas)

- [ ] Implementar PWA offline mode
- [ ] Pub/Sub pattern para notifications
- [ ] Email delivery fallback (SendGrid)
- [ ] Multi-region Cloud Functions
- [ ] Firestore backup + disaster recovery

Fase 3 (8-12 semanas)

- ☐ Load testing a 50K usuarios
- ☐ Performance optimization
- ☐ Security audit externo
- ☐ Compliance audit (GDPR, Ley 25.326)
- ☐ Documentation + runbooks

---

8. CONCLUSIÓN

VIABILIDAD: ✅ 100% VIABLE

PESSY puede escalar a **50,000 usuarios en 1 año** con:

- **Costo estimado:** $1,455/mes (50K usuarios) = **$0.029/usuario**
- **Break-even:** ~$0.99/mes/usuario (7% de precio típico)
- **Créditos disponibles:** $5,300-$7,000 (primeros 12 meses)
- **Arquitectura:** Solid Firebase stack, escalable con mejoras

Prioridades críticas:

1. ✅ Automatizar deployment (CI/CD)
2. ✅ Caching strategy (reducir reads)
3. ✅ Image compression (reducir storage)
4. ✅ Monitoring + alertas
5. ✅ GDPR compliance

Riesgos manejables:

- Rate limiting, sharding, multi-region deployment
- Cloud Tasks para heavy workloads
- Fallback email provider
- Vector search para búsqueda

Recomendación: Ejecutar Plan Fase 0 inmediatamente, solicitar créditos Google, target launch Q3 2026.